UFC VIRTUAL · TRANSIÇÃO DIDÁTICA

Documentação técnica do sistema

Como o código funciona: linguagens, organização das pastas, painel, Editor Estrutural e o banco de dados (Supabase).

ATENÇÃO: o Supabase é TEMPORÁRIO e não será usado na versão final.

Sumário

1. Visão geral do sistema
2. Linguagens e tecnologias
3. Organização das pastas
4. Como o painel funciona (`src/`)
5. Como o Editor Estrutural funciona
6. **Banco de dados: Supabase (TEMPORÁRIO)**
7. Segurança e permissões
8. Preparando a versão final (trocando o Supabase)
9. Como rodar, testar e publicar

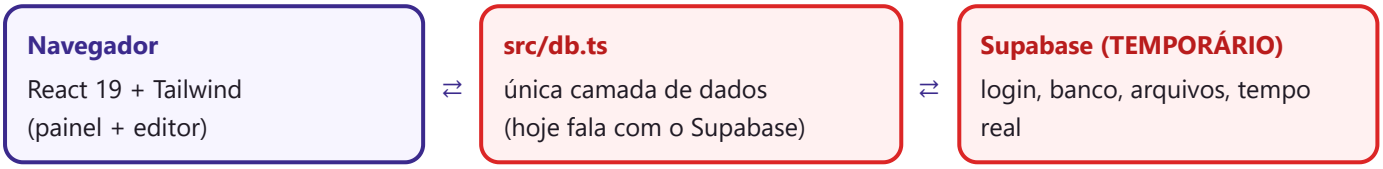
⚠ O SUPABASE É TEMPORÁRIO

Todo o armazenamento atual (login, dados, arquivos e imagens) está no Supabase apenas para desenvolvimento e validação. Ele NÃO será utilizado na versão final do sistema. O capítulo 6 descreve quais dados entram e saem dele e o capítulo 8 explica o que precisa ser trocado.

1. Visão geral do sistema

O sistema é uma **aplicação web (single-page application)** que roda inteiramente no navegador. Ele tem duas partes que trabalham juntas:

- **Painel de Transição Didática** (pasta `src/`): login, equipes, disciplinas, Kanban, tarefas, materiais, pendências/fórum e perfis de acesso (Administrador, Suporte técnico e Conteudista).
- **Editor Estrutural** (pasta `criacao de recursos/`): monta aulas com seções e componentes, sem escrever código, e exporta a aula como um site (.zip). Ele é embutido dentro do painel, na aba "Editor Estrutural" de cada disciplina.



Não existe servidor próprio: o site é estático (HTML, CSS e JavaScript) e hospedado no **GitHub Pages**. Tudo que precisa de servidor hoje (login, dados, arquivos) é atendido pelo Supabase.

2. Linguagens e tecnologias

Item	Uso
TypeScript / TSX	Painel: `App.tsx`, `db.ts`, `RecursosTab.tsx`, `supabase.ts`, `imageTools.ts`
JavaScript / JSX	Editor Estrutural e componentes de aula (pasta `criacao de recursos/`)
HTML e CSS	`app.html` (entrada), Tailwind CSS 4 para o visual, CSS próprio para KaTeX e código
SQL (PostgreSQL)	`supabase/schema.sql`: tabelas, regras de acesso (RLS), buckets e tempo real
TypeScript (Deno)	`supabase/functions/admin-users/index.ts`: função de servidor que cria usuários
React 19	Interface em componentes, com estado local (`useState`, `useEffect`, `useRef`)
Vite 8	Ferramenta de build e servidor de desenvolvimento
Tailwind CSS 4	Estilos por classes utilitárias
lucide-react / react-icons	Ícones
JSZip	Gera o .zip do site da aula (e embute as imagens)
KaTeX	Renderiza fórmulas (LaTeX)
highlight.js	Colore blocos de código
Vitest	Testes automatizados do Editor Estrutural
@supabase/supabase-js	Cliente do Supabase (**temporário**)

> Fontes usadas: Inter, Figtree e Anek Devanagari (pacotes @fontsource, empacotados no site — não dependem de internet).

3. Organização das pastas

```
TD-UFC-VIRTUAL/
├─ app.html                  página de entrada do painel (desenvolvimento)
├─ index.html + assets/      SITE COMPILADO (gerado por "npm run build"; GitHub Pages serve daqui)
├─ vite.config.js            configuração do Vite e atalhos (@ui, @lesson, @builder…)
├─ package.json              dependências e comandos do painel
├─ scripts/publish-root.mjs  copia o build para a raiz do repositório
├─ src/                      PAINEL
│   ├─ main.tsx              inicia o React
│   ├─ App.tsx               telas, perfis, Kanban, materiais, pendências, equipes (~1.700 linhas)
│   ├─ RecursosTab.tsx       aba "Editor Estrutural" (lista de aulas, envio e aprovação)
│   ├─ db.ts                 ★ ÚNICA camada de dados (todas as leituras/gravações)
│   ├─ supabase.ts           cliente do Supabase (TEMPORÁRIO)
│   ├─ imageTools.ts         reduz e converte imagens antes do envio
│   ├─ Brand.tsx             logos (IUVI/UFC)
│   └─ context.ts            contexto compartilhado com o editor
├─ supabase/                 BANCO TEMPORÁRIO
│   ├─ schema.sql            tabelas, RLS, buckets, tempo real
│   ├─ seed.sql / seed.local.sql   criação do administrador (o .local fica fora do Git)
│   ├─ patch-imagens.sql     ajuste do bucket de imagens
│   ├─ functions/admin-users/index.ts   função que cria/atualiza usuários
│   └─ LEIA-ME.md            passo a passo de configuração
├─ criacao de recursos/      EDITOR ESTRUTURAL (projeto próprio, com testes)
│   └─ src/
│       ├─ pages/Builder/    telas do editor (LessonBuilder, BlockCard, NestedBlocks,
│       │                    BlockPalette, RichTextEditor, LatexField, ImageField,
│       │                    ColorPicker, exportSite.js, blockHelp.js, blockExamples.js)
│       └─ components/
│           ├─ LessonRenderer/  como a aula aparece ao aluno (sectionCatalog.js, LessonRenderer.jsx,
│           │                    migrate.js, theme.js, spacing.js, math.js, highlight.js)
│           └─ ui/              componentes visuais (Callout, Quiz, Timeline, Table, Code…)
└─ docs/                     documentação em PDF
```

O painel "importa" o editor por meio de **atalhos de caminho** definidos no `vite.config.js` (`@lesson` , `@builder` , `@ui` …). Assim os dois projetos compartilham o mesmo código sem duplicar.

4. Como o painel funciona (`src/`)

4.1 Fluxo de uma sessão

1. `main.tsx` monta o `App`.
2. O `App` verifica se há sessão salva; se houver, busca o perfil do usuário (`fetchProfile`) e carrega os dados (`loadAll`).
3. Conforme o **perfil** (admin, tecnico, conteudista), a barra lateral mostra opções diferentes e as telas filtram o conteúdo.
4. Cada ação (criar tarefa, enviar material...) chama uma função do `db.ts`, depois recarrega os dados (`run` → `refresh`).
5. Alterações feitas por outras pessoas chegam em **tempo real** e atualizam a tela sozinhas.

4.2 Principais telas (todas dentro de `App.tsx`)

Tela	Quem usa	O que faz
Login	todos	Entrada por usuário (ou e-mail), com confirmação ao sair
Painel de Projetos	todos	Cartões das disciplinas visíveis ao perfil, com fase e pendências
Equipes	admin	Cria equipes, cadastra membros, altera senhas
Nova Disciplina / Gerenciar Professores	admin / técnico	Cadastra disciplinas e conteudistas
Kanban	todos (ações conforme o perfil)	Etapas, tarefas e conclusão de etapas
Materiais de Apoio	todos	Envio de arquivos/links e aprovação
Pendências e Fórum	todos	Pedidos de ajuste com histórico de respostas
Editor Estrutural	todos	Lista de aulas, criação, envio, aprovação (`RecursosTab.tsx`)

4.3 Etapas do processo

Cada disciplina passa por: `recebimento` → `design_educacional` → `producao` → `revisao` → `implantado`. Cada etapa tem tarefas e pode ser marcada como concluída.

4.4 Estados de uma aula

`rascunho` → `enviado` (para a Equipe EAD) → `ajuste` (com retorno escrito) ou `aprovado`.

4.5 Camada de dados: `db.ts`

Todas as telas conversam com os dados **somente** por funções de `src/db.ts` (`signIn`, `loadAll`, `createDiscipline`, `addTask`, `addMaterial`, `updateResource`...). O arquivo também converte os nomes das colunas do banco (`snake_case`) para o formato usado nas telas (`camelCase`). Isso é importante: **trocar o banco significa reescrever só este arquivo** (veja o capítulo 8).

5. Como o Editor Estrutural funciona

5.1 Modelo da aula

A aula é um objeto JSON:

```
{ aula: "1", titulo: "Aula 1 — …", objetivo: "…", cor: "#…",
  sections: [
    { type: "section", name: "Abertura",       // nome interno (o aluno não vê)
      label: "", title: "Introdução", subtitle: "…",   // visíveis ao aluno
      titleBefore: 12, titleAfter: 8,                  // espaçamentos em px
      children: [ { type: "intro", text: "…" }, { type: "image", src: "…" }, … ] } ] }
```

Toda aula tem **seções**; os **componentes** ficam sempre dentro de uma seção.

5.2 Peças do editor

Arquivo	Função
`sectionCatalog.js`	**Fonte da verdade**: lista de componentes, campos, grupos e valores padrão
`LessonBuilder.jsx`	Tela principal: barra da aula, seções, adicionar/arrastar componentes, salvamento automático
`BlockCard.jsx` / `NestedBlocks.jsx`	Cartões de edição de seção e de componente
`SchemaField.jsx`, `fieldInputs.jsx`	Geram os formulários a partir dos campos do catálogo
`RichTextEditor.jsx`, `LatexField.jsx`, `ImageField.jsx`	Texto formatado, fórmulas e envio de imagem
`LessonRenderer.jsx`	Transforma o JSON em página para o aluno (também usado na pré-visualização)
`migrate.js`	Converte aulas de formatos antigos para o atual
`exportSite.js`	Gera o .zip do site (HTML + CSS + JS + imagens)

5.3 Componentes disponíveis

Seção, bloco com texto, **título avulso**, imagem com legenda, vídeo, carrossel, grupo de acordeões, lista de cards, linha do tempo, tabela de dados, tabela comparativa, caixa de referência, conclusão, quiz, chamada (CTA), link de acesso, fórmula, formulário de fórmulas, exercício passo a passo, destaque (definição/teorema…), glossário, bloco de código, código em várias linguagens e quiz de código.

> **Para criar um componente novo:** (1) adicionar uma entrada em `sectionCatalog.js`; (2) adicionar o `case` correspondente em `LessonRenderer.jsx`; (3) opcionalmente, textos de ajuda em `blockHelp.js` e exemplo em `blockExamples.js`. O formulário de edição é gerado automaticamente.

5.4 Salvamento automático

Dentro do painel, cada alteração dispara um salvamento após um curto intervalo (*debounce*). O editor controla o estado (salvando / salvo / erro), repete a gravação se falhar, grava ao esconder a aba ou sair e

avisa se houver algo pendente ao fechar. Os dados da aula vão para a coluna `lesson` (JSON) da tabela `resources`.

5.5 Imagens e exportação

Ao enviar uma imagem, `imageTools.ts` reduz o maior lado a 1600 px e converte para WebP (GIFs ficam como estão). O arquivo vai para o armazenamento e o endereço público é guardado na aula. Ao "Baixar site", `exportSite.js` baixa essas imagens e as coloca dentro do .zip, de modo que o site da aula funcione offline.

6. Banco de dados: Supabase

DESTAQUE — LEIA PRIMEIRO

⚠ O SUPABASE É TEMPORÁRIO — NÃO SERÁ UTILIZADO NA VERSÃO FINAL

O Supabase foi adotado somente como solução provisória para desenvolver e validar o sistema (login, dados, arquivos e tempo real) sem precisar montar um servidor.

A **versão final usará outra infraestrutura**. Por isso: não trate o Supabase como definitivo, não construa novas dependências diretas dele fora de `src/db.ts` e planeje a migração dos dados (capítulo 8).

6.1 Serviços do Supabase em uso

Serviço	Para quê
Auth (login)	Entrada por e-mail/senha. O "usuário" digitado vira `usuario@painel.example.com`; o administrador entra com e-mail real.
Postgres (banco)	Tabelas do sistema, com regras de acesso por linha (RLS)
Storage (arquivos)	Dois "buckets": `materiais` (privado) e `aulas-imagens` (público)
Realtime	Avisa o painel quando alguém altera dados, para atualizar a tela
Edge Function `admin-users`	Criar usuários, trocar senhas e remover — usa a chave de serviço só no servidor

6.2 Dados guardados (tabelas)

Tabela	Dados principais
teams	id, nome da equipe
profiles	id (= usuário do login), nome, usuário, perfil (admin/tecnico/conteudista), equipe
disciplines	título, conteudista responsável, equipe, etapa atual, etapas concluídas (JSON), datas
tasks	disciplina, título, descrição, prazo, etapa, concluída, autor
materials	disciplina, nome/link, caminho do arquivo, tamanho, situação (aguardando análise / recebido / ajuste), remetente
pendencies	disciplina, texto, tipo (técnico→professor / professor→técnico), situação (aberta/resolvida), autor
pendency_comments	pendência, texto, autor
resources	disciplina, **aula completa em JSON**, estado (rascunho/enviado/ajuste/aprovado), retorno da equipe, autor, último editor

Ao excluir uma disciplina, tarefas, materiais, pendências, comentários e aulas dela são apagados em cascata. Os arquivos no armazenamento não são apagados automaticamente.

6.3 O que o sistema ENVIA ao Supabase e o que RECEBE

Ação no sistema	Enviado ao Supabase	Recebido do Supabase
Entrar	e-mail (montado a partir do usuário) e senha	sessão (token) e, em seguida, o perfil do usuário
Carregar o painel	consulta única (`loadAll`)	equipes, perfis e disciplinas com tarefas, materiais, pendências, comentários e aulas — já filtrados pelo perfil
Criar equipe	nome	equipe criada (id, nome)
Criar membro / trocar senha	nome, usuário, senha, perfil, equipe (via função `admin-users`)	confirmação ou mensagem de erro
Criar / excluir disciplina	título e conteudista / id da disciplina	id criado / confirmação
Kanban	etapa atual, etapas concluídas, tarefas (título, descrição, prazo, concluída)	dados atualizados (e aviso em tempo real aos demais)
Enviar material	arquivo (bucket `materiais`) + nome, tamanho, remetente	registro criado; link temporário (1 h) para baixar
Pendências e fórum	texto, tipo, autor; respostas; marcar como resolvida	pendências com histórico de respostas
Editor Estrutural (salvar)	aula em JSON, estado, retorno da equipe	confirmação; versão salva
Enviar imagem da aula	imagem reduzida (WebP/JPG/PNG/GIF, até 5 MB) no bucket `aulas-imagens`	endereço público da imagem
Tempo real	(inscrição nas tabelas)	aviso “algo mudou” → o painel recarrega os dados

6.4 Chaves e segredos

- **No site (público):** `VITE_SUPABASE_URL` e `VITE_SUPABASE_PUBLISHABLE_KEY` (arquivo `.env`). Podem ficar no navegador porque a proteção é feita pelas regras de acesso do banco.
- **Nunca no site:** a chave *secret / service_role*. Ela só existe dentro da Edge Function, no servidor.
- `supabase/seed.local.sql` (contém a senha inicial do administrador) está no `.gitignore` e nunca vai para o repositório.

7. Segurança e permissões

Perfil	O que enxerga e pode fazer
Administrador	Tudo, de todas as equipes: cria equipes, membros, disciplinas; altera senhas; exclui disciplinas
Suporte técnico	Só disciplinas e conteudistas da **própria equipe**: Kanban, aprovações, pendências, cadastro de conteudistas, download do site da aula
Conteudista	Só as **próprias** disciplinas: envia materiais, abre pendências, monta e envia aulas (não aprova nem baixa o site)

- As regras são aplicadas **no banco** (RLS), não apenas escondendo botões na tela. Mesmo alterando o código no navegador, o usuário não acessa dados de outra equipe.
- Funções auxiliares (`auth_role` , `auth_team` , `can_see_discipline` , `is_staff_of_discipline`) concentram a lógica de "quem pode o quê".
- O cadastro aberto de usuários deve ficar **desligado**; só o administrador (ou técnico, para conteudistas da própria equipe) cria contas, pela Edge Function.
- Arquivos de `materiais` são privados e acessados por links assinados temporários; imagens das aulas são públicas porque aparecem no site baixado.

> Como o banco é temporário, estas regras (RLS) precisarão ser **reimplementadas no novo back-end** (verificação de perfil e equipe em cada requisição).

8. Preparando a versão final (trocando o Supabase)

> **⚠ LEMBRETE: SUPABASE = TEMPORÁRIO**
>
> Esta é a lista do que precisa ser substituído quando o sistema for para a infraestrutura definitiva.

8.1 O que depende do Supabase hoje

Onde	Dependência	O que fazer
`src/supabase.ts`	Cliente e chaves	Remover; criar o cliente do novo back-end
`src/db.ts`	Todas as leituras/gravações, login, arquivos	**Reescrever as funções** mantendo os mesmos nomes e formatos de retorno
`src/App.tsx`	Sessão de login e tempo real (`supabase.auth` , `supabase.channel`)	Trocar por login e atualização (tempo real ou consulta periódica) do novo sistema
`src/RecursosTab.tsx`	Envio de imagens (via `uploadLessonImage`)	Apontar para o novo armazenamento
`supabase/`	Esquema SQL, função de usuários, regras RLS	Recriar o esquema no novo banco; reescrever `admin-users` como rota protegida do servidor
Imagens já enviadas	Endereços públicos do Supabase gravados dentro das aulas (JSON)	Migrar os arquivos e reescrever os endereços nas aulas

8.2 Passos sugeridos

1. Definir o novo back-end (banco, login, armazenamento de arquivos, API).
2. Implementar as mesmas operações listadas no capítulo 6.3 e as regras de acesso do capítulo 7.
3. Reescrever `src/db.ts` (a interface das funções permanece; as telas não mudam).
4. Trocar login e tempo real no `App.tsx` .
5. Exportar os dados do Supabase (tabelas e arquivos) e importar no novo sistema; ajustar os links das imagens nas aulas.
6. Remover a pasta `supabase/` , a dependência `@supabase/supabase-js` e as variáveis `VITE_SUPABASE_*` ; **desativar o projeto no Supabase**.

> O Editor Estrutural (pasta `criacao de recursos/`) **não depende do Supabase**: ele só recebe funções do painel (salvar, enviar imagem). Pode ser mantido como está.

9. Como rodar, testar e publicar

Objetivo	Comando (na pasta do projeto)
Instalar dependências	`npm install`
Rodar em desenvolvimento	`npm run dev` (abre o painel; precisa do arquivo `.env`)
Gerar e publicar o site	`npm run build` → gera `dist/` e copia `index.html` + `assets/` para a raiz; depois `git push`
Testes do Editor Estrutural	`cd "criacao de recursos"` e `npx vitest run`

9.1 Publicação

O GitHub Pages está configurado para servir a branch `main` a partir da raiz. Como o site compilado fica na raiz (`index.html` e `assets/`), basta rodar `npm run build`, fazer o commit e o push.

9.2 Variáveis de ambiente (`.env`)

```
VITE_SUPABASE_URL=https://SEU-PROJETO.supabase.co      (temporário)
VITE_SUPABASE_PUBLISHABLE_KEY=chave-publica            (temporário)
```

9.3 Boas práticas ao alterar o código

- Novas leituras/gravações de dados devem passar por `src/db.ts`.
- Novos componentes de aula: sempre pelo catálogo (`sectionCatalog.js`) + renderizador.
- Nunca colocar chaves secretas ou senhas no código ou no Git.
- Rodar os testes do Editor Estrutural e o `npm run build` antes de publicar.

⚠ RESUMO FINAL

O código do painel e do Editor Estrutural é a base do sistema final. **O Supabase é só um apoio temporário** e será substituído; todo o acesso a ele está concentrado em `src/db.ts`, `src/supabase.ts`, no login/tempo real do `App.tsx` e na pasta `supabase/`.